

Project Design Phase: Problem – Solution Fit

Header	Details
Date	02 October 2026
Team ID	
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	2 Marks

Problem – Solution Fit Canvas

1. Customer Segment(s) (CS)

- **Target Users:** ServiceNow System Administrators, Security Administrators, IT Service Management (ITSM) Process Owners, and Data Compliance Auditors.
- **Key Persona:** System Administrator managing sensitive departmental data across multiple branches (e.g., ECE, EEE, CSE) within an enterprise instance.

2. Jobs-To-Be-Done / Problems (J&P)

- **Jobs-To-Be-Done:**
 - Enforce row-level and field-level security access policies dynamically at runtime.
 - Restrict user access to records based on dynamic record field values (e.g., Branch == 'EEE').
 - Maintain centralized security governance across Create, Read, Write, and Delete operations.
- **Core Problems:**
 - Out-of-the-box (OOTB) static role-based access control (RBAC) grants access based solely on assigned roles without checking record field context.
 - Users from unauthorized branches/departments can view or modify records containing sensitive information.

3. Triggers (TR)

- **Internal Triggers:** Security audit findings revealing unauthorized data exposure between departments.
- **External Triggers:** New regulatory compliance guidelines requiring strict data isolation per department/branch.

- **Operational Triggers:** Onboarding new users/roles (e.g., bb1, bb2, bb3, bb4) requiring restricted permissions.

4. Emotions: Before / After (EM)

- **Before:**
 - Anxious and uncertain about potential compliance breaches and unauthorized record edits.
 - Overwhelmed by trying to manage record-level security manually or through fragmented workaround rules.
- **After:**
 - Confident and reassured knowing that server-side scripts dynamically block unauthorized access across all forms, lists, and APIs.
 - In control with explicit audit traces confirming 100% security compliance.

5. Available Solutions (AS)

- **Alternative 1: Separate Tables per Department**
 - *Pros:* Simple data separation.
 - *Cons:* Highly unscalable, multiplies database maintenance, breaks unified reporting.
- **Alternative 2: Before Query Business Rules**
 - *Pros:* Filters records from list views.
 - *Cons:* Does not natively secure individual field edits (Write/Delete) or direct API calls as robustly as the platform Security Engine.
- **Alternative 3: UI Policies / Client Scripts**
 - *Pros:* Easy to set up on forms.
 - *Cons:* Insecure; only hides fields on the UI and can easily be bypassed via browser dev tools or REST APIs.

6. Customer Constraints (CC)

- **Performance Budget:** Evaluation scripts must introduce negligible execution latency (< 2ms per transaction) so platform navigation remains fast.
- **Role Dependencies:** Requires elevating to security_admin privilege to configure and activate Access Control Lists (ACLs).
- **Complexity:** Needs clear logic to prevent locking administrators out of critical system records.

7. Behaviour (BE)

- **Direct Actions:**
 - Configures Advanced ACL rules using current object evaluations and gs.hasRole() checks.
 - Elevates security privileges (security_admin) before modifying access control rules.
 - Tests access policies by impersonating test users assigned specific custom roles (bb1, bb2, bb3, bb4).
- **Indirect Actions:**
 - Monitors System Logs and ACL Debuggers to trace evaluation passes/fails.

8. Channels of Behaviour (CH)

8.1 Online

- **ServiceNow Filter Navigator:** System Security → Access Control (ACL).
- **Instance Test Interfaces:** u_institution_details.list / User Impersonation.

8.2 Offline

- Security policy reviews and team architectural sign-off meetings.

9. Problem Root Cause (RC)

- Standard ServiceNow Access Controls evaluate static role assignments at the table level by default. When business requirements demand conditional, attribute-based access control (ABAC)—such as filtering by a record's Branch field—static roles alone fail, necessitating custom scripted evaluation within the ACL Engine.

10. Your Solution (SL)

- **Script-Controlled ACL Architecture on u_institution_details Table:**
 1. **Read ACL:** Requires role bb1 and evaluates a script checking if `gs.hasRole('admin')` or `current.u_branch == 'EEE'`.
 2. **Create ACL:** Requires role bb2 to permit new record creation.
 3. **Write ACL:** Requires role bb3 to allow record modifications.
 4. **Delete ACL:** Requires role bb4 to authorize record deletion.
- **Outcome:** Replaces static access controls with dynamic, field-value-sensitive security rules that enforce complete data isolation across all platform interfaces with minimal performance impact.